TAO TrafficCamNet traffic scene object detection

TAO TrafficCamNet traffic scene object detection

1. Case goal and processing flow
2. The checks before running it
3. Start the detection
4. The real-time detection with a USB camera
5. The real-time detection with a CSI camera
6. Frequently asked questions
7. References

> Applicable environment: Jetson Orin NX, JetPack 7.2.1, DeepStream 9.1.

1. Case goal and processing flow

TrafficCamNet is aimed at the road traffic frames; it detects the traffic objects such as the vehicles and the pedestrians frame by frame, and it outputs the category, the confidence and the bounding box. The course configuration adds an NvDCF Tracker after the detection, so the text such as `car 42` or `person 2` in the frame carries the track ID inside the current video; the ID is not the real personal identity. This case does not recognize the characters of the license plate; for the license plate recognition, refer to the LPR case of this chapter.

```
H.264 MP4 → hardware decoding → TrafficCamNet TensorRT inference → the detection result is overlaid
                                                                  ├→ the desktop/VNC preview
                                                                  └→ the internal sink output
```

The course example is wrapped by the DeepStream `deepstream-app` + the TAO configuration. The model does the detection; the decoding, the drawing of the boxes and the preview are done by the DeepStream pipeline.

2. The checks before running it

Please operate in the graphical desktop of the Jetson or in a VNC terminal that is connected to the desktop. A pure SSH session usually has no usable graphical display environment.

```
cd ~/deepstream_tao_apps/deepstream_app_tao_configs

# Look at the default video that is referenced in the configuration file
grep -A5 "\[source0\]" run_trafficcamnet.txt

# Confirm that the model/engine path is valid
grep -E "onnx-file|model-engine-file|config-file" run_trafficcamnet.txt
```

Confirm that the input video exists and that the path of the configuration file is valid. The default video is under the DeepStream samples.

3. Start the detection

For the local video detection, run the following in the terminal:

```
export DISPLAY=:0 && deepstream-app -c
~/deepstream_tao_apps/deepstream_app_tao_configs/run_trafficcamnet.txt
```



The script loads the TrafficCamNet configuration and the TensorRT engine and then starts the DeepStream pipeline. When it runs normally, the terminal keeps printing the state/performance information of the pipeline, and a preview with the detection boxes appears on the Jetson desktop or in the VNC.

The rectangular boxes in the frame indicate the position of the target as judged by the model, the category text is the predicted category and the number at the end is the Tracker ID. The occlusion, the lighting, the size of the target and the shooting angle all affect the detection result; an occasional missed detection or false detection does not mean that the startup failed.

The default input file is referenced by the configuration, and it is the sample that comes with DeepStream:

```
file:///opt/nvidia/deepstream/deepstream-9.1/samples/streams/sample_720p.h264
```

When you replace the test video, edit the `uri=file://` of the `[source0]` section in `run_trafficcamnet.txt` to the absolute path of the target MP4.

4. The real-time detection with a USB camera

The USB mode streams from the camera in real time and displays the detection results, which makes it suitable for verifying the scene on site.

After connecting the USB camera, confirm that the system has detected the device:

```
ls -l /dev/video*
v4l2-ctl -d /dev/video0 --list-formats-ext
```



The node, the pixel format and the resolution of the other cameras can be different. You have to make sure that no other program is using that camera.

Start it in a Jetson desktop/VNC terminal:

```
export DISPLAY=:0 && ~/deepstream_tao_demo/run_usb.sh trafficcamnet
```

Note: the parameter name `trafficcamnet` in `run_usb.sh` is `trafficcamnet` (and not `trafficcamnet_usb`); the script automatically composes the `run_trafficcamnet_usb.txt` configuration file.

Normally the real-time preview opens and the detection boxes are overlaid in the frame. A USB camera is a live source, so it does not naturally play back to the EOS like an offline MP4; the current entry point is used for the real-time preview.

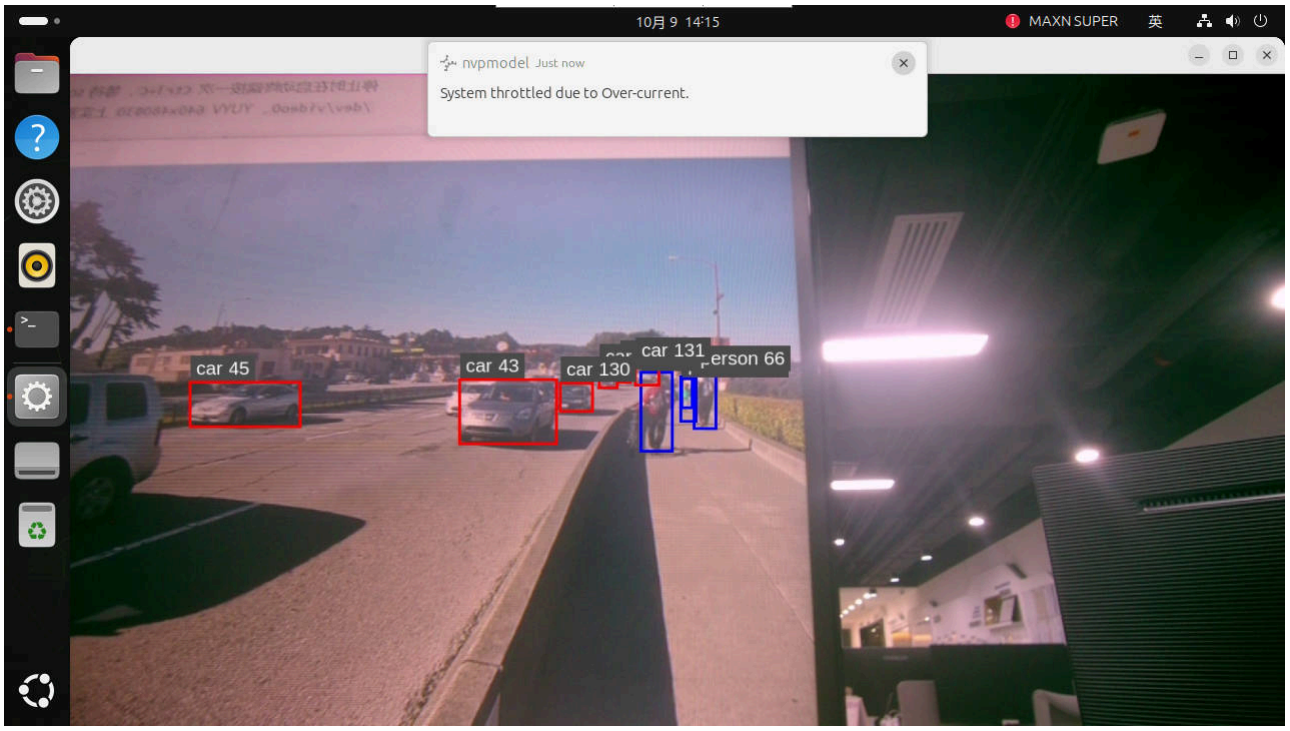
To stop it, press `Ctrl+C` once in the terminal that started it, wait for the source EOS and `App run successful`, and confirm that the preview has closed and that the camera device has been released before you start another camera program.

The script queries the formats that the camera actually supports and generates a temporary `640x480@15` configuration with the priority. The older version was measured at about 15 FPS; the exact resolution and the frame rate are the ones in the `USB source` of the startup log.

5. The real-time detection with a CSI camera

CSI uses the Jetson Argus camera link, not the USB/V4L2 path of `/dev/video0`. Before starting it you have to connect a supported CSI camera, confirm that the sensor driver and the Argus service work normally, and operate in the graphical session of the Jetson desktop/VNC.

```
~/deepstream_tao_demo/run_case.sh trafficcamnet csi
```



Whether the sensor is available depends on the specific carrier board, the ribbon cable, the camera module and the device tree/driver configuration. If the startup fails, first check the camera connection, the sensor driver and the Argus log, and then look at the first error of DeepStream; do not take the result of the USB `/dev/video0` format check as evidence that the CSI works normally.

To exit, press `Ctrl+C` once in the terminal that started it and wait for the application and the camera streaming to exit before you start another program.

6. Frequently asked questions

Symptom	The check and the handling
`Failed to load config file`	Confirm that you start it from the `~/deepstream_tao_apps/deepstream_app_tao_configs` directory; or check the absolute path of `config-file=` in the configuration file.
The input file cannot be found or decoded	Confirm that the MP4/H.264 file that the `uri=file://` of `[source0]` points to exists; check the path with `ls`.
There is no preview window	Confirm that the command was started in the graphical session of the Jetson desktop/VNC; check whether `DISPLAY=:0` is set.
`Could not open ... for writing`	Confirm that the output path in the sink configuration exists, that there is enough disk space and that the current user has the write permission.
It reports that the engine does not match the device/version	The engine is related to the GPU, to TensorRT, to CUDA, to the precision and to the batch configuration. After deleting `models/*.engine`, the first run rebuilds it automatically (about 3-5 minutes).

Symptom	The check and the handling
The tracker user-meta warning appears	It is a known warning of this case and it does not block the pipeline; if the pipeline is still running and the EOS is received normally, accept the result of the complete chain.
The USB camera cannot be opened	Confirm that the device nodes such as `/dev/video0` exist, that the format is supported, that the user has the access permission and that the device is not used by another process.
The USB streams but there is no preview	Confirm that you started it in the graphical desktop/VNC session; look for a camera format negotiation or a display sink error.
The CSI startup reports that the source cannot be found or that there is no image	The script directly uses the Argus `type=5`; check the sensor connection, the driver, `nvargus-daemon` and whether the camera is in use.
It reports `nvv4l2h264enc has no property named 'gpu-id'`	It is a known non-blocking warning of DS 9.1; it appears when the OSD sink is initialized and it does not affect the inference or the display.

When you troubleshoot, keep the complete startup command and the log around the first error; do not delete the engine, modify several configurations or reinstall DeepStream directly.

7. References

- The NVIDIA DeepStream TAO application examples: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/tao_apps
- The DeepStream TAO integration documentation: https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_TAO_integration.html